PRÁTICAS EM SISTEMAS OPERACIONAIS

AULA 6

Prof. André Luiz de Souza Paula

CONVERSA INICIAL

Sistemas Operacionais GNU/Linux

Objetivo: compreender como instalar e remover *software* nos sistemas operacionais GNU/Linux, *scripting* básico, conceitos de virtualização e agendamento de tarefas

Temas de estudo:

1. Sistemas de gerenciamento de pacotes
2. Instalação, atualização e remoção de pacotes
3. Conceitos básicos de virtualização
4. *Scripting* básico
5. Agendamento de tarefas

TEMA 1 – SISTEMAS DE GERENCIAMENTO DE PACOTES

Os sistemas operacionais GNU/Linux permitem basicamente duas formas de se instalar pacotes:

a) através das ferramentas de instalação;

b) ou manualmente compilando o pacote.

Existem muitas ferramentas de manipulação de pacotes e, quando estudamos instalação ou desinstalação em qualquer distribuição GNU/Linux, sempre encontramos o termo *pacote* em vez de *programa*. Esse termo representa o empacotamento de programas, que é um tipo especial de arquivo que contém os arquivos e a estrutura de onde cada arquivo deve ser gravado no sistema para que ele funcione corretamente, junto com um conjunto de controles e instruções que são executadas antes (*preinst*) e após (*postinst*) a instalação e remoção deles.

1.1 O que é um pacote de *software*

Pacote é um arquivo que, além de conter os arquivos necessários para instalação de determinado aplicativo, contém as informações de controle necessárias para que o gerenciador de pacotes possa instalar, manter e remover programas, incluindo suas dependências.

Cada distribuição GNU/Linux possui uma extensão representativa para seus pacotes. No Debian GNU/Linux e seus derivados, a extensão utilizada é ".deb". Outra extensão comumente encontrada é ".rpm", que é utilizada nas distribuições derivadas do RedHat GNU/Linux. Encontramos arquivos binários, como acontece quando instalamos alguns programas proprietários.

- **Dependências:** uma das características de sistemas GNU/Linux é a modularização de seus componentes, como também o uso intensivo de bibliotecas compartilhadas, ocorrendo também nos pacotes de aplicativos. Com isso, ao se instalar determinado pacote, quase sempre é necessário primeiro instalar outro pacote que contenha os "pré-requisitos" para a sua instalação e configuração. Por exemplo:

```
# apt-get install extremetuxracer
 Lendo listas de pacotes... Pronto
 Construindo árvore de dependências
 Lendo informação de estado... Pronto
 Os pacotes extra a seguir serão instalados:
 extremetuxracer-data libmikmod2 libsdl-mixer1.2 libsmpeg0
 tcl8.5
 Os NOVOS pacotes a seguir serão instalados:
 extremetuxracer extremetuxracer-data libmikmod2 libsdl-
 mixer1.2 libsmpeg0 tcl8.5
 0 pacotes atualizados, 6 pacotes novos instalados, 0 a serem
 removidos e 0 não atualizados.
 É preciso baixar 30,3MB de arquivos. Você quer continuar
 [S/n]?
```

1.2 Advanced Packet Tool (APT)

O Advanced Packet Tool (*APT*) é a principal ferramenta de manipulação de pacotes do Debian. Entretanto, pode ser considerado um *front-end* para o *Debian Package* (utilitário *dpkg)*. Com ele, é possível instalar, remover, reinstalar, configurar e atualizar pacotes.

É uma ferramenta completa que possui muitos recursos que são utilizados através dos vários comandos do APT. Os comandos mais utilizados são o "apt-get" e o "apt-cache". O *APT* possui uma ferramenta com interface gráfica que

possui as mesmas características e se chama “Gerenciador de Pacotes *Synaptic*”.

Antes de listarmos as principais opções do *APT,* devemos fazer algumas observações sobre algumas características do seu uso:

- Com exceção dos recursos de pesquisa (*search*), todos os demais só podem ser executados pelo usuário *root*. Quando usamos esses recursos com usuários comuns é apresentado este erro: “não foi possível abrir arquivo de trava */var/lib/dpkg/lock* – open (13 Permissão negada)”.
- Não é possível executar mais de uma instância ao mesmo tempo. Quando uma instância estiver sendo executada, as tentativas de se iniciar uma outra resultaram nesse erro: “Não foi possível obter trava /var/lib/dpkg/lock – *open* (11 Recurso temporariamente indisponível)”. É normal esse erro acontecer na seguinte situação: algum usuário deixa o *synaptic* aberto, ou está efetuando as atualizações de segurança, e tentar instalar algum pacote através do APT em linha de comando.
- Para funcionamento do APT, os repositórios de onde serão baixados os pacotes devem estar configurados no arquivo “*/etc/apt/sources.list*”.

A configuração do arquivo “*/etc/apt/sources.list*” deve ser feita da seguinte forma:

a) Primeiro deve ser passado o tipo de pacote que será instalado, com parâmetro “deb”. Se alterado para “deb-src”, define os pacotes que contêm os códigos-fonte dos pacotes tipo “deb”.
b) Em seguida, devemos definir qual protocolo será utilizado para transferências dos pacotes (http ou ftp).
c) O endereço (ip ou nome) do repositório e do compartilhamento desejado.
d) Após o endereço, devemos definir a qual *release* os pacotes devem pertencer e as seções desejadas (*main*, *contrib* e *non-free*).

Normalmente, quando instalamos um pacote, para que funcione corretamente, ele depende de outros pacotes (que contém bibliotecas ou outros programas). Nós chamamos esses pacotes de dependências. Um pacote pode ter mais de uma dependência, sendo que pacotes diferentes podem vir a ter outras dependências em comum e as próprias dependências podem vir a ter

dependências, como num grande conjunto de pacotes que unidos formam um sistema funcional.

Toda vez que um pacote possua dependências ao tentarmos instalá-lo, para que ele funcione, precisamos instalar também todas as suas dependências, como também as dependências de suas dependências. Isso torna esse processo complexo e trabalhoso para se fazer manualmente.

O *APT* foi umas das primeiras ferramentas para gerenciamento de pacotes. A ordem seguida por ele é esta: verificar se o pacote está disponível, resolve as dependências, realiza o *download* no servidor de todos os pacotes necessários, descompacta, configura e instala.

Quando o *APT* encontra as dependências já instaladas, verifica se estão com a versão atualizada e compatível. Caso não haja nenhum conflito com os pacotes instalados e suas dependências sejam sanadas, será exibido um relatório com toda relação de pacotes a serem instalados, removidos e atualizados e, em alguns casos, mostra uma relação de pacotes sugeridos ou recomendados para instalação junto a uma solicitação de confirmação para continuidade do processo.

Agora estudaremos algumas formas de uso do comando apt mostrando seus principais parâmetros:

```
~# apt update
```

Uma lista que contém uma descrição de todos os pacotes disponíveis para o Debian GNU/Linux, disponível dentro de cada repositório de pacotes. O “apt” consulta essa lista para saber o que pode ser instalado. Esse comando faz uma comparação entre a lista que é armazenada anteriormente e a lista que está no servidor de pacotes. Se a sua for mais velha, faz o *download* da mais recente. A atualização de um único pacote faz a atualização para uma nova lista. Esse comando sempre deve ser executado antes de se fazer a instalação de qualquer pacote ou após alterar o arquivo “*sources.list*”.

```
~# apt upgrade
```

Esta é a opção para atualização de pacotes. Quando se executa esse comando, uma lista de pacotes é copiada do repositório para a máquina local do usuário, e é feita uma comparação entre as versões de *software* existente no repositório e na máquina local. Caso existam versões mais diferentes para os

programas que o usuário possui em seu sistema, então, o APT atualizará esses programas no computador do usuário. Não é preciso dizer qual pacote você quer atualizar, independentemente de ser um ou mais, será feita toda atualização necessária.

```
~# apt clean
```

Após a instalação de um pacote, o seu arquivo .deb é armazenado em "*/var/cache/apt/archives*". No processo de instalação, o *APT* não remove os pacotes baixados. Se não remover os pacotes baixados, começará a ocupar espaço no disco rígido. Com o tempo, isso pode causar um problema de espaço. O comando "*apt clean*" faz a remoção de todos os pacotes do diretório de cache do APT.

```
~# apt install <NOME DO PACOTE>
```

Esse comando é executado para fazer instalação de pacotes no sistema. Se for necessário instalar mais de um programa, basta escrever todos os programas separados por espaço na linha de execução do comando. Caso existam dependências, elas são mostradas a você e o programa aguarda a sua confirmação ("s/n") para continuar. Existem vários motivos para ele solicitar por uma confirmação:

a) Uma lista de dependências muito grande, você pode não querer instalar todos os pacotes.
b) Pode não haver espaço em disco suficiente para instalar o programa e/ou suas dependências.
c) O pacote ser incompatível com outro já instalado e exigir que este seja removido.
d) O repositório não é uma fonte confiável para o Debian, que acontece quando não existe na máquina a chave pública do repositório de onde vem a atualização.
e) há mais de uma versão disponível para o mesmo pacote. Podemos querer instalar a versão mais antiga, tendo que executar o comando da seguinte forma:

```
~# apt install nome=2.0-0
```

~# apt dist-upgrade

Possibilita a atualização total da distribuição utilizada. É importante que o usuário saiba que a execução desse comando num sistema que utilize meta-pacotes customizados, ou versões de *software* instaladas manualmente, poderá gerar problemas graves, impossibilitando inclusive o uso do sistema. Para a utilização desse comando, é necessário alterar o arquivo de configuração do APT, o "/etc/apt/sources.list", configurando este para utilizar os repositórios da distribuição mais atual que será instalada.

~# apt remove <NOME DO PACOTE>

Remove um pacote. Essa operação faz a desinstalação de um pacote, mantendo o arquivo ".deb" do diretório de cache. Os arquivos de configuração e dados do pacote são mantidos. Se você remover um pacote por acidente, toda configuração dele ainda estará preservada.

~# apt --purge remove <NOME DO PACOTE>

Remove um ou mais pacotes e seus respectivos arquivos de configuração e dados. Neste caso, não há recuperação.

~# apt source <NOME DO PACOTE>

Permite que o código-fonte de um programa seja baixado para máquina local no lugar do binário do programa e seus scripts de controle, que compõem o pacote do programa propriamente. Para baixar arquivos contendo o código-fonte de programas para sua máquina, é necessário configurar adequadamente o arquivo "/etc/apt/sources.list" com a opção *deb-src*.

~# apt build-dep <NOME DO PACOTE>

Esse comando tenta resolver as dependências que um determinado pacote contendo código-fonte de um programa possui, para ser compilado. Por exemplo, se você deseja recompilar o código-fonte do "*pacman*" (um jogo). Se você executar o comando "apt build-dep pacman", o APT irá instalar todos os pacotes que sejam necessários para compilação do pacote "*pacman*". Em algumas situações, é necessário instalar outros pacotes manualmente.

```
~# apt check
```

É uma forma de fazer um diagnóstico. Ele atualiza o *cache* e verifica se há alguma dependência quebrada e necessita ser resolvida (utilize "*~# apt -f install*").

```
~# apt install --reinstall <NOME DO PACOTE>
```

Se for necessário reinstalar um pacote já presente no sistema. Para isso, podemos utilizar este comando:

```
~# apt search <NOME DO PACOTE>
```

Para procurar por um pacote, executamos o comando com opção "*Search*". Ele faz uma pesquisa na lista de pacotes baixados no comando "*apt update*", localizando pacotes que possuam a expressão fornecida no argumento "NOME DO PACOTE". A saída apresenta todos os pacotes que contenham a palavra fornecida como argumento em parte do nome ou na descrição do pacote.

```
~# apt show <NOME DO PACOTE>
```

Se for necessário visualizar todas as informações sobre o pacote, bem como seu mantenedor, sua versão, sua descrição, entre outras informações relevantes.

1.3 YellowDog Updater Modified (YUM)

O YUM é um gerenciador de pacotes em linha de comando para distribuições baseadas em *RPM* (*Red Hat Package Manager*). É uma ferramenta que permite atualizações automáticas e verificação de dependências. Os arquivos de configuração do repositório YUM estão localizados no diretório /etc/yum.repos.d/ e devem ter extensão ".repo".

As opções *install*, *search* e *remove* tem a mesma função do gerenciamento com APT.

1.4 Yet Another Setup Tool (YAST)

YAST é uma ferramenta de instalação e configuração para as distribuições openSUSE e SUSE Linux Enterprise. O objetivo é a facilidade de

uso, interface gráfica e capacidade de customizar rapidamente seu sistema durante e após a instalação. Entretanto, é possível usar em linha de comando.

As opções *install*, *search* e *remove* têm a mesma função do gerenciamento com APT.

TEMA 2 – INSTALAÇÃO, ATUALIZAÇÃO E REMOÇÃO DE PACOTES

O APT, YAST e YUM são ferramentas de *front-end*, ou seja, devem ser utilizadas para manter a integridade do sistema e utilização dos repositórios. Entretanto, essas ferramentas internamente utilizam os comandos *dpkg* e *rpm.*

2.1 *Debian Package* (DPKG)

O *Debian Package* (*DPKG*) é o comando responsável pela instalação, configuração e remoção de pacotes em sistemas Debian GNU/Linux. Sua execução é feita em linha de comando e funciona através de comando em ambiente de texto. O *DPKG* é usado por usuários avançados do Debian GNU/Linux e desenvolvedores para construção de pacotes.

O *dpkg* é usado principalmente como ferramenta de obtenção de informações sobre os pacotes instalados, na criação, na extração e na resolução de problemas com pacotes. Para instalação de pacotes, utilizamos o *APT*, que, como já estudado, executa o *dpkg* para efetivar a instalação. Os passos seguidos por são:

a) desempacota os arquivos debian;
b) executar o *script* de *preinst*;
c) copia todos os arquivos nos seus devidos lugares;
d) executar o *script* de *postinst.*

Os arquivos de configuração e os *scripts* de instalação (*preinst e postinst*) ficam dentro do diretório */var/lib/dpkg/info/<nome do pacote>*. Os *scripts* utilizam a linguagem Shell, na grande maioria das vezes.

O *dpkg* é composto por ferramentas que desempenham funções específicas em relação aos pacotes Debian GNU/Linux. A seguir, uma breve descrição do que as principais ferramentas fazem:

- dpkg – Desempenha a instalação, configuração e remoção de pacotes.
- dpkg-source – Empacota e desempacota os códigos-fonte de um .deb.

- dpkg-deb – Empacota e desempacota pacotes binários em um .deb.
- dpkg-reconfigure – Permite ajustes nos pacotes já instalados. É o "reconfigurador de pacotes". É uma ferramenta muito utilizada.

Utilizando o dpkg

Sintaxe do comando *dpkg*:

```
~# dpkg <opções> <ações> <pacotes>
```

As principais ações são:

- **-i <nome do pacote>** – Instala um pacote no sistema sem resolver dependências, podendo criar uma inconsistência no gerenciamento de pacotes.
- **--unpack <nome do pacote>** – Desempacota um arquivo .deb, extraindo o conteúdo e montando uma estrutura de diretórios e arquivos de acordo com a estrutura que foi usada no seu desenvolvimento. Observe que não será descompactado em qualquer lugar, mas, sim, no lugar pré-definido no pacote. Se o administrador verificar com o "*dpkg -l*" o *status* do pacote, verificará que o pacote possuirá o *status* "**U**". O que indica que o pacote foi somente descompactado e não instalado da forma correta.
- **-X <nome do pacote> <diretório>** – Faz a mesma coisa que o comando anterior, porém, em vez de extrair os arquivos nos seus devidos lugares, ele permite que o usuário escolha um novo destino. A diferença principal está no fato de o --unpack realizar a criação dos arquivos na barra (/) e o -X iniciar a partir do diretório corrente, ou que seja especificado. Poderemos, ainda, executar um com digitando o endereço completo (absoluto) do executável, ou colocando o binário dentro da vaiável de ambiente *PATH* do usuário que o irá executar.
- **--configure <nome do pacote>** – Realizará a configuração de todos os pacotes parcialmente instalados nos sistemas (ou seja, que deram erro por algum motivo durante a instalação). Durante a instalação, algum erro pode vir a ocorrer, fazendo com que o *dpkg* interrompa a instalação. Utilizando essa opção, na maioria das vezes, os erros são resolvidos. Os problemas ocorrem normalmente por algo não relacionado aos *scripts* de instalação, mas fatores externos, como falta de espaço em disco ou queda

de energia. Se o problema for causado pelos próprios *scripts* de configuração, o administrador terá que editá-los (*preinst* e *postinst*).

- **-r <nome do pacote>** – Remove um pacote do sistema mantendo os arquivos de configuração deste, se existirem.
- **-P <nome do pacote>** – Faz a remoção completa de um pacote do sistema, incluindo arquivos de dados, de configuração etc.
- **-C** – Checagem de pacotes mal instalados. Procura por pacotes que foram instalados incorretamente, quebrados, e estão parcialmente em seu sistema, indicando o que pode ser feito para que o problema seja corrigido.
- **--get-selections <padrão>** – Procura na lista de pacotes instalados no sistema pela expressão digitada através do argumento "<padrão>", ou, no caso deste não ser informado, todos os pacotes que já passaram pelo sistema. Os pacotes que estejam com o *status* "*install*" indicam os pacotes instalados, enquanto os que possuírem *status "deinstall"* os já desinstalados.
- **--print-architecture** – Exibe a arquitetura padrão utilizada pelos pacotes instalados no sistema.
- -l **<nome do pacote>** – Lista todos os pacotes instalados no sistema com uma descrição curta, podendo ter diferentes *status*: instalado corretamente com o *apt*; ter sido apenas desempacotado; ter tido sua instalação interrompida durante a instalação. Opcionalmente, poderá especificar o nome de um ou mais pacotes para verificar informações adicionais.
- **-s <nome do pacote>** – Lembre-se desse "**s**" como **show**. Exibe as informações sobre um determinado pacote, podendo este estar já instalado ou então ter sido instalado uma vez na vida. Essas informações são retiradas do arquivo "/var/lib/dpkg/available".
- **-l <nome do pacote>** – Exibe uma série de informações sobre um pacote instalado. Sempre indique o nome completo. Se o administrador tentar utilizar apenas parte do nome do pacote, o erro "Arquivo ou diretório não encontrado" será apresentado na tela. As informações apresentadas são próximas com as do parâmetro "-s", porém, o pacote não precisa ter sido instalado, e temos uma informação adicional: o tamanho do arquivo de

controle e uma descrição e quanto ocupa de espaço (*conffiles*, *control*, *md5sums*, *postinst*, *preinst*, *posrm* e *prerm*).

- **-S <nome do arquivo>** – Permite descobrir qual pacote instalou o arquivo. Pode ser muito interessante, já que o administrador pode utilizar essa opção em um computador para descobrir qual pacote instalar se desejar ter esse arquivo. Para saber o nome do pacote, basta ir até o computador que possui o arquivo e verificar, com essa opção, aquele pacote que o possui.

Como já foi dito, ele pode ser usado para qualquer arquivo, não somente um arquivo executável (ex.: *sources.list*). O resultado será mostrado da seguinte forma:

```
pacote: linha que contém o arquivo
***(diretório também é um arquivo)
```

- **-L <nome do pacote>** – Mostra todos os arquivos dentro de seus respectivos diretórios. Veja que o arquivo de configuração apt.conf do pacote *apt* para ser alocado dentro do diretório "/etc/apt", precisa criar o diretório apt. Então a ordem será:

```
/
/etc
/etc/apt
/etc/apt/apt.conf
```

Desta forma, o administrador consegue verificar exatamente onde está cada um dos diretórios e arquivos criados na instalação do pacote.

Caso um pacote seja removido, esse comando terá sua fonte atualizada e mostrará apenas aqueles arquivos e diretórios que ainda existem. Siga esse exemplo: instale o pacote tuxpaint, utilize esse parâmetro (*-L*), remova com o *dpkg -r* e então repita a saída do comando com o parâmetro "-L". Poderá observar que somente os arquivos de configuração permanecerão no computador.

- **-c <nome do pacote>** – O retorno é o mesmo que do -L1, com a diferença que o -c verifica todos os diretórios e arquivos que criados na instalação do pacote, diferente do -L que verifica quais arquivos e diretórios existem,

ou seja, de aplicações instaladas. Ele é interessante para que o administrador verifique se a instalação irá criar algum diretório ou arquivo no lugar de um outro já existente. As opções dpkg tanto podem ser especificadas através da linha de comando, como através do arquivo de configuração do *dpkg* (*/etc/dpkg/dpkg.cfg*). As principais opções são:

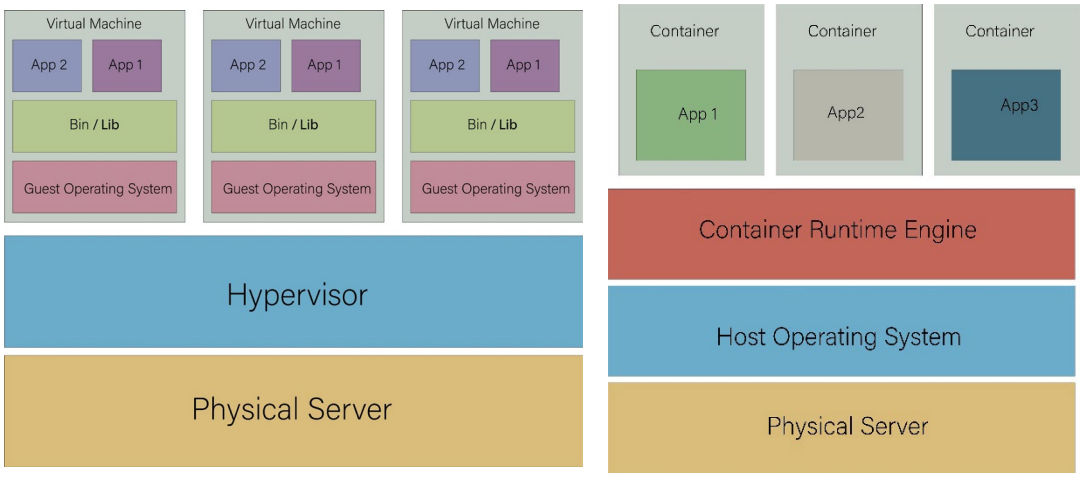
- **-B** – O *dpkg* não garante que outros pacotes ainda instalados dependam do que foi removido. Essa opção faz com que esses pacotes também sejam automaticamente removidos do sistema.
- **--no-act** – Faz um teste de acordo com as ações solicitadas. Por exemplo, podemos simular a instalação ou remoção de um pacote para verificar se ocorre algum erro no sistema de gerenciamento de pacotes.
- **--log=arquivo** – Permite definir um arquivo em que o dpkg vai armazenar os registros das suas operações. Por padrão o arquivo "/var/log/dpkg.log" recebe essas informações.

2.2 *Red Hat Package Manager* (RPM)

O *Red Hat Package Manager* (RPM) é um sistema de gerenciamento de pacotes para Linux. O RPM permite ao administrador instalar, remover e obter informações sobre pacotes. Com o RPM, é possível reparar um banco de dados danificado, construir pacotes a partir de arquivos-fonte, verificar a assinatura digital de pacotes RPM, simular instalação, entre outros.

TEMA 3 – CONCEITOS BÁSICOS DE VIRTUALIZAÇÃO

A virtualização é uma técnica utilizada para criar um ambiente virtual, com objetivo de executar *desktops (VDI)*, servidores, áreas de armazenamento, redes e outros recursos. O *hardware* apresentado para esses ambientes é, na verdade, uma abstração criada por *software* e limita as funções do *hardware* físico. É uma forma para executar várias máquinas virtuais simultaneamente em uma única máquina física. A figura a seguir representa dois tipos de virtualização:



Crédito: Tea Talk/shutterstock; Tea Talk /shutterstock

Para que isso seja possível, existe o *Hypervisor*, que podemos definir como um sistema operacional modificado e especializado na execução de máquinas virtuais.

Os servidores virtuais criados pela virtualização criam um ambiente similar a de um servidor físico e otimizam o uso de recursos porque estão independentes do *hardware*. Podemos ainda entender de forma simples como sendo o particionamento de um servidor físico em vários servidores lógicos, e a função desse particionamento é do *Hypervisor*.

Apesar de parecer algo novo, e largamente utilizado em ambientes de nuvem, não é um conceito novo porque, desde a introdução do "*TimeSharing*", que compartilha CPU para execução de várias tarefas, o desenvolvimento do compartilhamento do *hardware* só evoluiu.

3.1 Hypervisores Nativo ou Bare Metal (Tipo 1)

O *Hypervisor* é instalado diretamente no *hardware* e não requer a instalação prévia de um sistema hospedeiro. Os recursos de *hardware* são acessados de forma direta. Melhor *performance* e escalabilidade, mas o suporte ao *hardware* é limitado.

Tipo 1: *OpenStack, VMWare (ESX e ESXi), Citrix Systems XenServer, Microsoft Hyper-V, Red Hat Enterprise Virtualization (KVM)*

3.2 Hosted hypervisor (Tipo 2)

É baseado no uso de um sistema operacional hospedeiro, as chamadas de *hardware* dependem exclusivamente do hospedeiro. Maior compatibilidade, aumento *overhead* e menor *performance.*

Tipo 2: *Oracle VirtualBox, VMWare Workstation, Microsoft Virtual, KVM, "Docker"*

TEMA 4 – *SCRIPTING* BÁSICO

Este tópico tem o objetivo de descrever como criar um *script* básico para automatização de tarefas no sistema operacional GNU/Linux.

4.1 Ambiente SHELL

Ambiente Shell é uma interface textual que tem a função de interpretar os comandos disponíveis em um sistema operacional e enviar para o núcleo (kernel) do sistema, retornando posteriormente o resultado da execução do comando.

4.2 Variáveis em Shell Script (Bash)

As variáveis podem ser classificadas em variáveis de ambiente e variáveis de sessão (usuário). As variáreis de ambiente são definidas em dois arquivos principais: */etc/profile* e */etc/enviroment*. Vamos listar a seguir as principais variáveis:

- **BASH:** contém a informação sobre a qual SHELL está sendo executado.
- **HOSTNAME:** contém o nome do computador em uso.
- **USER:** contém o nome do usuário da sessão atual.
- **PATH:** mantém a lista dos diretórios com arquivos executáveis, onde são encontrados os comandos enviados ao SHELL.
- **PS1:** mantém a configuração do *prompt* de comando de 1º nível.
- **PS2:** mantém a configuração do *prompt* de comando de 2º nível.
- **DISPLAY:** exibe a informação do número do terminal gráfico.
- **SHELL:** exibe o SHELL que está sendo executado.

- **HISTFILE:** configura o arquivo que deve receber o histórico de comandos digitado na última sessão.
- **HISTFILESIZE:** limita o tamanho máximo do arquivo de histórico de commandos.

4.2 Exemplos de criação de *scripts* básicos

1) Script "Alo mundo" – Apenas mostra uma mensagem na tela.

```
~# chmod 755 helloworld.sh
~# ./helloworld.sh
Ola mundo...
```

```
#!/bin/bash
# Script Hello World <Criado em 08/12/2023>
# Por.: André Luiz <aluiz@mail.com>

export MSG="Ola mundo..."
echo $MSG

exit 0
```

Nesse exemplo, destaca-se que uma variável, em Shell Script, não precisa ser declarada e pode ser usada para qualquer tipo de dados (inteiro, string, boolean, entre outras). Entre o nome da variável e o valor que está sendo atribuído, não pode haver espaços, somente o caráter igual "=".

2) *Script* "Contador de pares" – Mostra uma sequência de números pares de zero até o número informado.

```
~# chmod 755 Contar.sh
~# ./Contar.sh 20
0 2 4 6 8 10 12 14 16 18 20
```

```
#!/bin/bash
# Script Contador <Criado em 09/12/2023>
# Por.: André Luiz <aluiz@mail.com>

export NUMERO=$1
export SEQUENCIA=$(seq 0 2 $NUMERO)
echo $SEQUENCIA

exit 0
```

Observe, nesse exemplo, o uso de um comando do sistema (*seq*), o que indica que o Shell Script permite que sejam automatizados comandos normalmente digitados em um terminal por um usuário. A saída de processamento desse comando é armazenada na variável “SEQUENCIA” e pode ser utilizada durante a execução do *script*.

3) Script “Encontre” – Valida se um determinado arquivo pode ser encontrado no sistema.

```
~#./encontre.sh /etc/locale.conf
/usr/share/man/man5/locale.conf.5.gz
O arquivo locale.conf existe.
```

```
#!/bin/bash
# Script encontre <Criado em 08/12/2023>
# Por.: André Luiz <aluiz@mail.com>

FILE="$1"
locate $FILE && echo "O arquivo $FILE existe." && exit 0
echo "O arquivo $FILE NÃO foi encontrado!" && exit 1
```

Esse *script* ainda não utiliza a estrutura de decisão “if” (se isso, faça aquilo), contudo, contém um recurso muito utilizado no Shell Script para tomada de decisão: ***&&***. A sintaxe do uso desse recurso está exemplificada a seguir:

```
<Comando a ser executado> && <Se o comando anterior foi executado com sucesso faça este comando>
```

Outro recurso utilizado nesse *script* é uma resposta para o sistema operacional com relação a sua execução: `exit`

Quando o comando ***exit*** recebe o valor zero, ele retorna para o sistema operacional informando que o *script* terminou e foi executado com sucesso. Qualquer outro valor é compreendido pelo sistema operacional como sendo um código de erro.

4) *Script* “Usuário” – Valida se um determinado usuário está cadastrado.

```
~# chmod 755 usuario.sh
~# ./usuario.sh
Informe o nome do usuário a ser consultado: Bill
```

O usuário Bill ***NÃO*** *é um usuário* ***cadastrado****.*

```
#!/bin/bash
# Valida Usario <Criado em 09/12/2023>
# Por.: André Luiz <aluiz@mail.com>

read -p "Informe o nome do usuário a ser consultado: " LOGIN
id $LOGIN && echo "O usuário $LOGIN é um usuário cadastrado." && exit 0
echo "O usuário $LOGIN NÃO é um usuario cadastrado." && exit 1
```

Neste script, foi utilizado o comando *read*, que tem a função de interagir com o seu usuário e solicitar uma *string*, ou um número, que será armazenado em uma variável (neste caso a variável LOGIN).

5) *Script* "Meu Computador" – Exibe a configuração atual do computador.

```
~#./meu_computador.sh
Descrição do Sistema
Sistema operacional: Debian GNU/Linux
Processador: Intel(R) Core(TM) i5-7200U CPU @ 2.50GHz
Clock da CPU (MHz): 3100.0000
Memória da Máquina: 7.9G
Placa de vídeo: HD Graphics 620
Placa de Rede: RTL810xE PCI Express Fast Ethernet
Placa de Som:  Sunrise Point-LP HD Audio
Este computador está ligado por: 20 min
```

```
#!/bin/bash
# Por.: André Luiz aluiz@mail.com
# Script Hello World <Criado em 09/12/2023>

export OS=$(lsb_release -d | cut -d: -f2)
export CPU=$(lscpu  | grep  "Model name" | cut -d: -f2)
export CLOCK="$(lscpu  | grep  "CPU max MHz" | cut -d: -f2)"
export MEM=$(lsmem | grep 'Total online' | cut -d: -f2)
export VGA=$(lshw -class display 2>/dev/null | grep product| cut -d: -f2)
export NET=$(lshw -class net 2>/dev/null | grep product| cut -d: -f2)
export SOUND=$(lshw -class sound 2>/dev/null | grep product| cut -d: -f2)
export UPTIME=$(uptime | cut -d" " -f4,5)

echo "Descrição do Sistema
Sistema operacional: $OS
Processador: $CPU
Clock da CPU (MHz): $CLOCK
Memória da Máquina: $MEM
Placa de vídeo: $VGA
Placa de Rede: $NET
Placa de Som: $SOUND
Este computador está ligado por: $UPTIME" | sed "s/  //g"

exit 0
```

Esse script possui vários comandos que exibem a configuração do computador e, para filtrar apenas os valores desejados, as saídas são filtradas usando os comandos:

a) *grep*: usado para selecionar as linhas que contém uma determinada expressão.
b) *cut*: corta a linha encontrada, separando apenas a coluna desejada.

TEMA 5 – AGENDAMENTO DE TAREFAS

Um sistema operacional é composto por vários serviços, entre eles executar rotinas rotineiras e repetitivas. A ferramenta compatível com o Agendador de Tarefas do MS-Windows, no ambiente GNU/Linux, é o CRON.

5.1 Cron

O CRON é um serviço carregado durante o processo de *boot* que possibilita a programação de execução de processos e comandos para serem realizadas periodicamente.

É um recurso fundamental para automatização da administração do sistema e em outros casos em que se faz necessário o agendamento de execução de uma determina atividade.

5.2 Automatização de *Scripts* (tarefa)

O agendamento de tarefas pode ser realizado pelo usuário *root* para as tarefas do sistema, ou por usuário comum para tarefas que exijam menos privilégio. O arquivo */etc/crontab* contém as tarefas agendadas para o sistema e o usuário pelos comandos do item 5.3.

A seguir a sintaxe do arquivo /etc/contrab:

(veja também: *~$ man 5 crontab*)

```
52 18 1 * * root /usr/bin/backup.sh
|  |  | | | |      \_Comando que será executado
|  |  | | |  \_ UID que executará o comando
|  |  | |  \_ Dia da semana (0--7)
|  |  |  \_ Mês (1--12)
|  |   \_ Dia do Mês (1--31)
|   \_ Hora (0--23)
 \_ Minuto (0--59)
```

5.3 Comandos de manipulação do Cron

Os comandos a seguir vão auxiliar na manutenção das tarefas agendadas do cron:

1) O arquivo crontab é o principal arquivo de configuração do agendador de tarefas (cron), cada linha representa uma tarefa agendada:

```
~# cat /etc/crontab
```

2) Após qualquer alteração no arquivo */etc/crontab,* é necessário reiniciar o serviço que controla a execução das tarefas:

```
~# systemctl restart cron
```

3) Quando houver necessidade de validar quais foram as tarefas agendadas pelo usuário, basta utilizar a opção “-l” do comando *crontab*:

```
~# crontab -l
```

4) Se for necessário adicionar, alterar ou remover uma tarefa agendada, utilize a opção “-e”:

```
~# crontab -e
```

5) Para limpar todas as tarefas agendadas por um usuário de uma só vez, execute a opção “-r”:

```
~# crontab -r
```

FINALIZANDO

Com o objetivo de fixar conhecimento sobre os principais termos utilizados nesta etapa, vamos listar a seguir os principais conceitos a serem revisados:

- **BARE METAL:** o termo *Bare Metal* é muito utilizado para definir um servidor físico. Inclusive existem serviços de nuvem que permitem “*alocation*”, que é uma forma de permitir que o cliente leve seu próprio servidor para um *datacenter*, alocando espaço. Entretanto, no contexto desta etapa, nos referimos a Bare Metal como um tipo de virtualização em que o *software* de virtualização é instalado diretamente no *hardware* (servidor físico).
- **DEPENDÊNCIAS:** nesta etapa, utilizamos alguns termos relativos ao empacotamento, que é uma técnica de preparar um *software* pré-compilado para instalação em um sistema operacional. As dependências são as bibliotecas e outros *softwares* necessários para o correto funcionamento do pacote que se deseja instalar.
- **HYPER-V:** é um produto de virtualização da Microsoft, funciona como um serviço dos seus sistemas operacionais. Permite a criação de máquinas virtuais e pode ser definido como um hypervisor tipo 2, que, apesar de funcionar em um sistema hospedeiro, está totalmente integrado ao MS-Windows e permite escalabilidade.
- **HYPERVISOR:** é uma camada de *software* que permite abstração entre um *hardware* hospedeiro (servidor físico), funciona como um sistema operacional modificado para suportar a criação de *containers* ou máquinas virtuais.

- **KVM:** da mesma forma que o Hyper-V transforma um MS-Windows Server tradicional em um Hypervisor, o KVM é capaz de transformar uma instalação padrão de um sistema GNU/Linux baseada em Kernel. Ele permite que uma máquina anfitriã (hospedeira/*host*) execute ambientes virtuais isolados (Máquinas Virtuais - VMs).
- **OPENSTACK:** o OpenStack extrapola a função de Hypervisor, é um conjunto de *software* que permite a administração de uma estrutura de nuvem de código aberto. Lembrando que a virtualização é a base para os serviços de Nuvem.
- **PACOTE DE *SOFTWARE*:** um pacote de *software* no contexto desta etapa se refere a um aplicativo pré-compilado, preparado e testado para a execução em um determinado sistema operacional. Um pacote, além de conter o código binário e algumas bibliotecas, controla a sua instalação e permite melhor controle pelo sistema operacional.
- **SYNAPTIC:** interface gráfica (*frontend*) para o gerenciador de pacotes APT, muito utilizado em distribuições para usuário final, como Ubuntu e outras distribuições que têm por objetivo permitir que um usuário comum possa gerenciar seus aplicativos.

REFERÊNCIAS

DEITEL, P. J.; DEITEL, H. M.; CHOFFNES, D. R. **Sistemas Operacionais**. 3. ed. Pearson, 2017. BVP. Disponível em: <https://plataforma.bvirtual.com.br/Acervo/Publicacao/315>. Acesso em: 19 dez. 2023.

HILL, B. M.; BACON, J. **O Livro Oficial do Ubuntu**. 2. ed. Porto Alegre: Bookman, 2008.

LINUX.ORG. **Linux**. Disponível em: <https://www.linux.org/>. Acesso em: 19 dez. 2023.

MACHADO, F. B.; MAIA, P. L. **Fundamentos de Sistemas Operacionais**. LTC, 2011. (BVMB)

NEMETH, E.; SNYDER, G.; HEIN, T. R. **Manual Completo do Linux**: guia do administrador. 2. ed. São Paulo: Pearson Prentice Hall, 2007. (BVP)

SILBERSCHATZ, A.; GALVIN, P. B.; GAGNE, G**. Fundamentos de Sistemas Operacionais**. 9. ed. Rio de Janeiro: LTC, 2015. (BVMB)

TANENBAUM, A. S. **Sistemas operacionais modernos**. 4. ed. São Paulo: Pearson Prentice Hall, 2008.